

OpsFlow

Documento de visão, arquitetura e planejamento técnico



Plataforma SaaS para gestão operacional de ordens de serviço, clientes, equipes, estoque, faturamento e monitoramento operacional.

Objetivo	Criar um sistema de estudo realista para praticar backend, arquitetura, mensageria, DevOps e operação ponta a ponta.
Arquitetura inicial	Core API em monólito modular + microserviço inicial de notificações com RabbitMQ.
Stack base	NestJS, TypeScript, PostgreSQL, Redis, RabbitMQ, Docker, Jest, Prometheus, Grafana e GitHub Actions.
Estratégia	Evolução por fases: fundação, núcleo operacional, operação real, performance, observabilidade e deploy.

Versão consolidada a partir do conteúdo fornecido pelo usuário

1. Visão geral

O OpsFlow foi concebido como um projeto de estudo suficientemente grande para ensinar arquitetura, backend, banco de dados, filas, cache, observabilidade, segurança, testes e deploy sem introduzir complexidade prematura.

A proposta central é evoluir um único produto por fases: começar com um monólito modular bem desenhado, adicionar mensageria cedo e incorporar um microserviço pequeno e isolado para aprendizado prático controlado.

2. Objetivo técnico do projeto

O sistema deve servir como laboratório prático para desenvolvimento de competências em:

- Modelagem de domínio e regras de negócio reais.
- Arquitetura modular com princípios de DDD leve, Hexagonal e Clean Architecture.
- API REST madura com autenticação, autorização, paginação, filtros e documentação.
- Banco relacional com foco em consistência, índices e performance.
- Uso de Redis para cache, rate limit, locks e idempotência.
- Mensageria com RabbitMQ, workers, retries e dead-letter queue.
- Observabilidade com logs estruturados, métricas e dashboards.
- Testes automatizados, Docker, CI/CD e deploy em nuvem.

3. Escopo do produto

3.1 Contexto de negócio

A plataforma simula uma empresa de manutenção técnica que atende clientes corporativos com contratos e SLAs. O sistema centraliza atendimento, execução, estoque, cobrança e qualidade operacional.

3.2 Usuários do sistema

- Administrador
- Supervisor
- Técnico
- Financeiro
- Cliente externo

3.3 Principais capacidades do produto

- Cadastrar clientes, unidades, usuários, equipes técnicas, contratos e SLAs.
- Abrir chamados, converter chamados em ordens de serviço e acompanhar todo o fluxo operacional.
- Controlar agenda, alocação de técnicos, reagendamentos e filas de pendências.
- Gerenciar peças, movimentação de estoque, reservas e baixas automáticas.
- Gerar cobrança, relatórios e indicadores operacionais.
- Auditar ações, emitir notificações e operar com observabilidade e deploy automatizado.

4. Diretriz arquitetural

A arquitetura recomendada para a V1 é um core principal em monólito modular, implementado em NestJS, acompanhado de um microserviço inicial de notificações. A comunicação entre os dois deve ser assíncrona, via RabbitMQ.

Essa abordagem reduz a complexidade operacional no início, mantém o domínio principal coeso e ainda expõe o projeto a contratos de evento, mensageria, observabilidade distribuída e consumidores idempotentes.

4.1 Decisão arquitetural

Componente	Responsabilidade	Justificativa
Core API	Domínio principal, autenticação, cadastros, chamados, ordens de serviço, estoque, billing e auditoria.	Mantém o núcleo de negócio centralizado e com boundaries claros entre módulos.
Notification Service	Consumo de eventos, registro de notificações internas, envio de e-mails e alertas futuros.	É o microserviço certo para começar: pequeno, útil, realista e fácil de isolar.
RabbitMQ	Transporte assíncrono de eventos e tarefas.	Permite aprender fila, retry, DLQ, contratos de evento e idempotência sem fragmentar o domínio cedo demais.

4.2 Aplicação dos princípios

- DDD leve: entidades, value objects, serviços de domínio quando necessários e linguagem de negócio consistente.
- Hexagonal: domínio e casos de uso não dependem do framework; persistência, mensageria e autenticação entram como adapters.
- Clean Architecture pragmática: organização por domain, application, infrastructure e presentation.

5. Estrutura sugerida do código

A estrutura abaixo favorece crescimento modular, isolamento de responsabilidades e facilidade de testes.

```
src/  
  modules/  
    auth/  
      domain/  
      application/  
      infrastructure/  
      presentation/  
    customer/  
    contract/  
    ticket/  
    work-order/  
    scheduling/  
    inventory/
```

billing/
audit/
dashboard/
shared/
domain/
application/
infrastructure/

apps/
core-api/
notification-service/
libs/
contracts/
shared-kernel/

6. Módulos funcionais

Módulo	Escopo resumido
Autenticação e controle de acesso	Login JWT, refresh token, logout, recuperação de senha, RBAC, permissões por módulo e auditoria de login.
Cadastro base	Clientes, unidades, usuários, equipes, catálogo de serviços, contratos e SLAs.
Chamados e ordens de serviço	Abertura, triagem, atribuição, status, histórico, comentários e anexos.
Agenda operacional	Agenda de técnicos, janelas de atendimento, conflito de horários, reagendamento e fila de pendências.
Estoque e peças	Cadastro de peças, movimentação, reservas, inventário, baixa automática e alerta de estoque mínimo.
Faturamento	Geração de cobrança, cálculo por contrato, desconto, taxa, multa, exportação e relatório mensal.
Notificações	E-mail transacional, notificação interna, eventos de domínio e processamento assíncrono.
Dashboard e indicadores	Tempo médio de atendimento, SLA, produtividade, peças mais usadas e faturamento por período.
Auditoria e compliance	Trilha de auditoria por entidade, histórico de alterações e exportação de logs críticos.

7. Requisitos não funcionais

- Performance: p95 de leitura inferior a 300 ms nas rotas principais, paginação obrigatória e uso de índices nas buscas críticas.
- Segurança: hash forte para senhas, rate limit em autenticação, validação de entrada, secrets fora do código e proteção contra OWASP Top 10.
- Observabilidade: logs em JSON, correlation id por request, métricas Prometheus, dashboards Grafana, healthcheck e readiness.
- Confiabilidade: retries controlados, dead-letter queue, consumidores idempotentes e rollback transacional onde necessário.
- Escalabilidade: separação entre API, worker e scheduler; containers independentes e possibilidade de horizontalizar workers.
- Manutenibilidade: monólito modular, boundaries claros, testes automatizados e ADRs para decisões relevantes.

8. Stack recomendada

Camada	Tecnologia
Core API	NestJS + TypeScript
Microserviço inicial	NestJS Microservices
Banco de dados	PostgreSQL
ORM	Prisma ou TypeORM
Cache e locks	Redis
Mensageria	RabbitMQ
Testes	Jest + testes de integração
Documentação	Swagger / OpenAPI
Infra local	Docker Compose + Nginx + Mailpit
Observabilidade	Prometheus + Grafana
CI/CD	GitHub Actions
Deploy inicial	VPS com Docker + Nginx
Frontend leve	Next.js ou React admin simples

9. Modelagem inicial de entidades

Entidades centrais sugeridas: User, Role, Permission, Customer, CustomerUnit, Contract, SLA, ServiceCatalog, Ticket, WorkOrder, Technician, Team, ScheduleSlot, InventoryItem, StockMovement, WorkOrderPart, Invoice, Notification e AuditLog.

Relacionamentos-chave:

- Customer possui muitas units e contracts.
- Contract define as regras de SLA.
- Ticket pertence a uma unidade do cliente e pode gerar uma WorkOrder.
- WorkOrder pertence a um ticket, pode ser atribuída a técnico ou equipe e pode consumir peças.
- WorkOrder concluída pode gerar Invoice.
- Entidades críticas devem possuir trilha de auditoria.

10. Endpoints principais

Auth

- POST /auth/login
- POST /auth/refresh
- POST /auth/logout

Clientes

- GET /customers
- POST /customers
- GET /customers/{id}
- PATCH /customers/{id}

Chamados

- POST /tickets
- GET /tickets
- GET /tickets/{id}
- PATCH /tickets/{id}/status
- POST /tickets/{id}/assign

Ordens de serviço

- POST /work-orders
- GET /work-orders
- GET /work-orders/{id}
- PATCH /work-orders/{id}/start
- PATCH /work-orders/{id}/complete
- POST /work-orders/{id}/parts

Estoque

- GET /inventory/items
- POST /inventory/items
- POST /inventory/movements

Billing

- POST /billing/invoices/generate
- GET /billing/invoices

Dashboard

- GET /dashboard/overview
- GET /dashboard/sla
- GET /dashboard/productivity

11. Eventos assíncronos e mensageria

Eventos iniciais recomendados:

- auth.user_logged_in
- ticket.created
- ticket.assigned
- work_order.created
- work_order.started
- work_order.completed
- sla.at_risk
- invoice.generated
- stock.low

Primeiro fluxo distribuído sugerido:

- Usuário faz login no Core API.
- Core API publica o evento auth.user_logged_in.
- Notification Service consome o evento.
- O consumidor registra notificação interna, log complementar de auditoria ou aciona envio de e-mail.

Conceitos que devem aparecer desde cedo: retry, dead-letter queue, deduplicação, consumidor idempotente e backoff simples.

12. Banco, cache e performance

- Migrations versionadas, seeds para ambiente local e constraints reais no banco.
- Índices nas colunas de busca e uso de EXPLAIN ANALYZE nas queries críticas.
- Cache Redis para dashboard e catálogos, além de rate limit de login e locks distribuídos.
- Consultas críticas: listagem de chamados, agenda de técnicos, dashboard de SLA, faturamento por mês e estoque mínimo.
- Evoluções futuras: materialized views, réplica de leitura e particionamento de auditoria/logs.

13. Testes, qualidade e segurança

- Pirâmide de testes com foco em regras de negócio, integração com banco/fila/cache e poucos E2E.
- Cobertura obrigatória de transições de status, regras de SLA, reserva de estoque, billing, autorização e idempotência.
- Lint, análise estática e testes na pipeline.

- JWT bem implementado, validação de payload, CORS e headers seguros, rate limit e auditoria de operações sensíveis.

14. DevOps e operação

Containers previstos:

- core-api
- notification-service
- worker
- scheduler
- postgres
- redis
- rabbitmq
- nginx
- prometheus
- grafana
- mailpit

Fluxo sugerido de CI/CD:

- Lint e análise estática.
- Testes unitários e de integração.
- Build de imagens.
- Scan básico.
- Merge na main para deploy em staging.
- Tag de release para deploy em produção.

Adoção de Kubernetes deve ficar para depois da consolidação de Docker, pipeline, observabilidade e operação em VPS.

15. Fases de entrega

Fase 1 — Fundação

Docker Compose, Core API em NestJS, Notification Service, PostgreSQL, autenticação, usuários/perfis, RabbitMQ, primeiro evento de login, OpenAPI e CI inicial.

Fase 2 — Núcleo operacional

Customers, contracts, tickets, work orders, histórico de status e testes de integração.

Fase 3 — Operação real

Agenda, equipes, estoque, reserva/baixa de peças e notificações por fila.

Fase 4 — Gestão e performance

Billing, dashboards, Redis, otimização de query e índices.

Fase 5 — Operabilidade

Logs estruturados, Prometheus, Grafana, alertas básicos e health checks.

Fase 6 — Deploy real

Staging, pipeline de deploy, domínio/subdomínio, reverse proxy e rollback básico.

16. Documentação obrigatória no repositório

- README.md
- docs/vision.md
- docs/requirements.md
- docs/architecture.md
- docs/adrs/
- docs/api.md
- docs/runbooks/
- docs/deployment.md
- docs/observability.md

17. Roadmap de estudo prático

- Sprint 1: setup do repositório, Docker Compose, Core API NestJS, Notification Service, autenticação, usuários/perfis, RabbitMQ e evento auth.user_logged_in.
- Sprint 2: clientes, unidades, contratos, SLA, seeds e factories.
- Sprint 3: tickets, fluxo de status e auditoria básica.
- Sprint 4: work orders, atribuição para técnico e histórico completo.
- Sprint 5: agenda, conflitos e testes de integração mais pesados.
- Sprint 6: estoque, reserva/baixa, transações e concorrência.
- Sprint 7: notificações, retries e DLQ.
- Sprint 8: faturamento, jobs agendados e exportação.
- Sprint 9: dashboard, Redis e otimização SQL.
- Sprint 10: Prometheus, Grafana, logs estruturados e health checks.
- Sprint 11: CI/CD, staging e deploy automatizado.
- Sprint 12: hardening de segurança, documentação final e refinamento arquitetural.

18. Recomendações finais

A ordem recomendada de evolução é: monólito modular, banco bem modelado, regras de negócio sérias, testes, filas, cache, observabilidade e deploy. Colocar Kubernetes antes disso tende a gerar sensação de avanço sem consolidar fundamentos.

O desenho final sugerido para a V1 é simples e forte: um Core API grande em NestJS, um microserviço pequeno de notificação, RabbitMQ conectando os dois e crescimento guiado por fases.

Esse formato ensina backend moderno, arquitetura séria, mensageria, microserviços, DevOps, deploy e observabilidade sem mergulhar cedo em um caos de múltiplos serviços.

19. Próximos passos sugeridos

- Definir oficialmente a V1 e congelar o escopo da Sprint 1.
- Escolher entre Prisma e TypeORM.

- Criar monorepo ou estrutura com apps/core-api e apps/notification-service.
- Montar o Docker Compose inicial com PostgreSQL, RabbitMQ, Redis, Mailpit e Nginx.
- Implementar autenticação, usuários/perfis e o evento auth.user_logged_in.
- Criar README, architecture.md e o primeiro ADR com a decisão de monólito modular + notification service.